

Application Security

Web App Development - Final Report

Students: Igor Świątek, Mikołaj Płatek
Student ID: 149547, 151911

Contents

1	Component Description	2
1.1	Purpose of the Component	2
1.2	Data Collected	2
2	Component Requirements	2
2.1	Functional Requirements	2
2.1.1	Registration Process	2
2.1.2	Login, Session Management & Password Recovery Process	2
2.1.3	Content Uploading & Role Based Access Control	3
2.2	Non-Functional Requirements	3
3	Component Architecture	4
3.1	Technology Stack	4
3.2	Architecture Diagram	4
4	Database Structure	5
5	UML Sequence Diagrams	6
6	Explanations of key implementation choices	12
7	Description of security mechanisms	13
8	Screenshots	14
9	Conclusion	20

1.0 Component Description

1.1 Purpose of the Component

The user registration module allows new users to create accounts by submitting personal information such as email, password and phone number. The module ensures secure handling of account creation, storage, and activation.

1.2 Data Collected

- Email address
- Password (securely hashed)
- Phone number
- Activation token

2.0 Component Requirements

2.1 Functional Requirements

2.1.1 Registration Process

1. **User Registration:** The system must present a form allowing a user to register.
2. **Input Validation (Client-Side):** The form must include client-side input validation and provide instructions for user to fill it correctly.
3. **Input Validation (Server-Side) & Error Handling:** The system must perform server-side input validation and provide appropriate error handling for invalid input.
4. **User Data Persistence:** The system must store user data in a database.
5. **Account Activation Token Generation:** The system must be able to generate unique account activation tokens.
6. **Account Activation Token Processing:** The system must be able to process and verify account activation tokens to activate a user's account.
7. **Password Strength Meter / Advanced Policy:** The registration form must include a visible password strength meter or enforce an advanced password policy (e.g., minimum length, required character types).
8. **Email Domain Restrictions:** The system must restrict email addresses based on a defined whitelist or blacklist of domains.
9. **CAPTCHA:** The system must implement CAPTCHA to prevent automated abuse.
10. **HTTPS:** The system must have HTTPS enforcing mechanism that is ready to use in production deployment.

2.1.2 Login, Session Management & Password Recovery Process

1. **User Login:** The system must present a form allowing a user to login.
2. **Password reset feature:** User must be able to reset password via link received in email.
3. **Input Validation (Client-Side):** The forms must include client-side input validation and provide instructions for user to fill it correctly.

-
4. **Input Validation (Server-Side) & Error Handling:** The system must perform server-side input validation and provide appropriate error handling for invalid input.
 5. **Session Management:** System must provide a secure session management and session destroying mechanism.
 6. **MFA:** System must provide a MFA mechanism.
 7. **Password Strength Meter / Advanced Policy:** The password change form must include a visible password strength meter or enforce an advanced password policy (e.g., minimum length, required character types).
 8. **CAPTCHA:** The system must implement CAPTCHA to prevent automated abuse.
 9. **HTTPS:** The system must have HTTPS enforcing mechanism that is ready to use in production deployment.
 10. **CSRF protection:** System must include CSRF tokens in all forms.

2.1.3 Content Uploading & Role Based Access Control

1. **Role Base Access Control:**
 - **Guest users (unauthenticated):** can browse public content and use keyword-based search
 - **Registered users (authenticated):** can add new content (including file uploads), comment / rate posts, and delete only their own content
 - **Administrators (authenticated):** can moderate the platform by deleting or managing any content item or comment
 - **Owners (authenticated):** have all the administrators' permissions and can manage users' roles
2. **Content Upload:** The system must allow users to upload their content
3. **Input Validation (Client-Side):** The forms must include client-side input validation and provide instructions for user to fill it correctly
4. **Input Validation (Server-Side):** The system must perform server-side input validation and provide appropriate error handling for invalid input
5. **Keyword Based Search:** The system must provide functionality to search posts based on keyword in the title
6. **CAPTCHA:** The system must implement CAPTCHA to prevent automated abuse.
7. **HTTPS:** The system must have HTTPS enforcing mechanism that is ready to use in production deployment.
8. **CSRF Protection:** System must include CSRF tokens in all forms.

2.2 Non-Functional Requirements

- **Secure Password Handling:** The system must securely handle passwords (hashing, salting, peppering).
- **Software Documentation:** Documentation must be prepared, including UML diagrams and architecture documentation.

-
- **Improved Password Hashing:** A robust or modern password hashing scheme must be implemented, extending the baseline secure password handling requirement.
 - **Clear Error Messages:** The system must provide clear and descriptive error messages during the sign-up process.
 - **Easy and Portable Server Setup:** The system must support an easy, portable deployment environment.

3.0 Component Architecture

3.1 Technology Stack

- Frontend: HTML/CSS/JS
- Server-Side Rendering: Jinja2
- Backend: Python/Flask
- Database: PostgreSQL
- Deployment: Docker, Docker Compose

3.2 Architecture Diagram

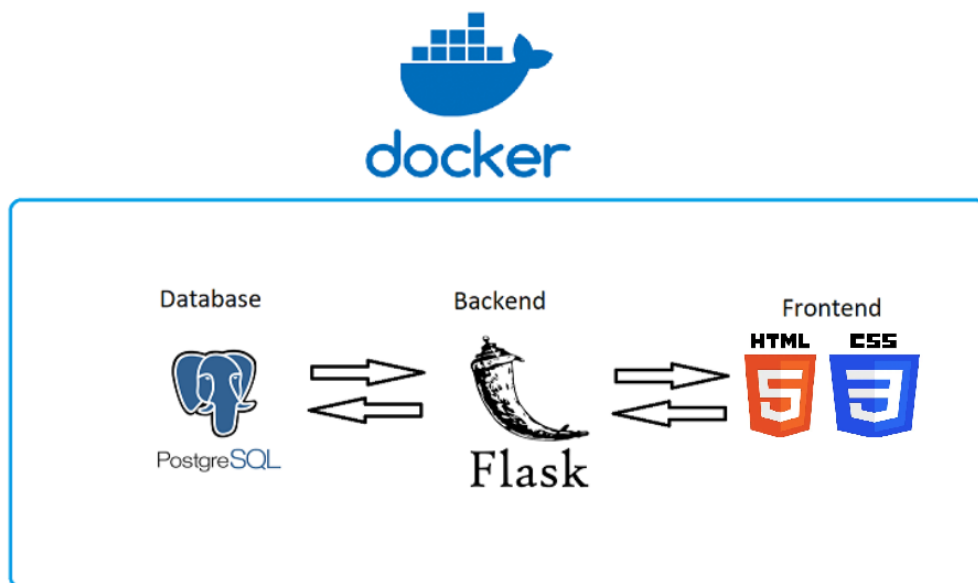


Figure 1: High-level Component Architecture

4.0 Database Structure

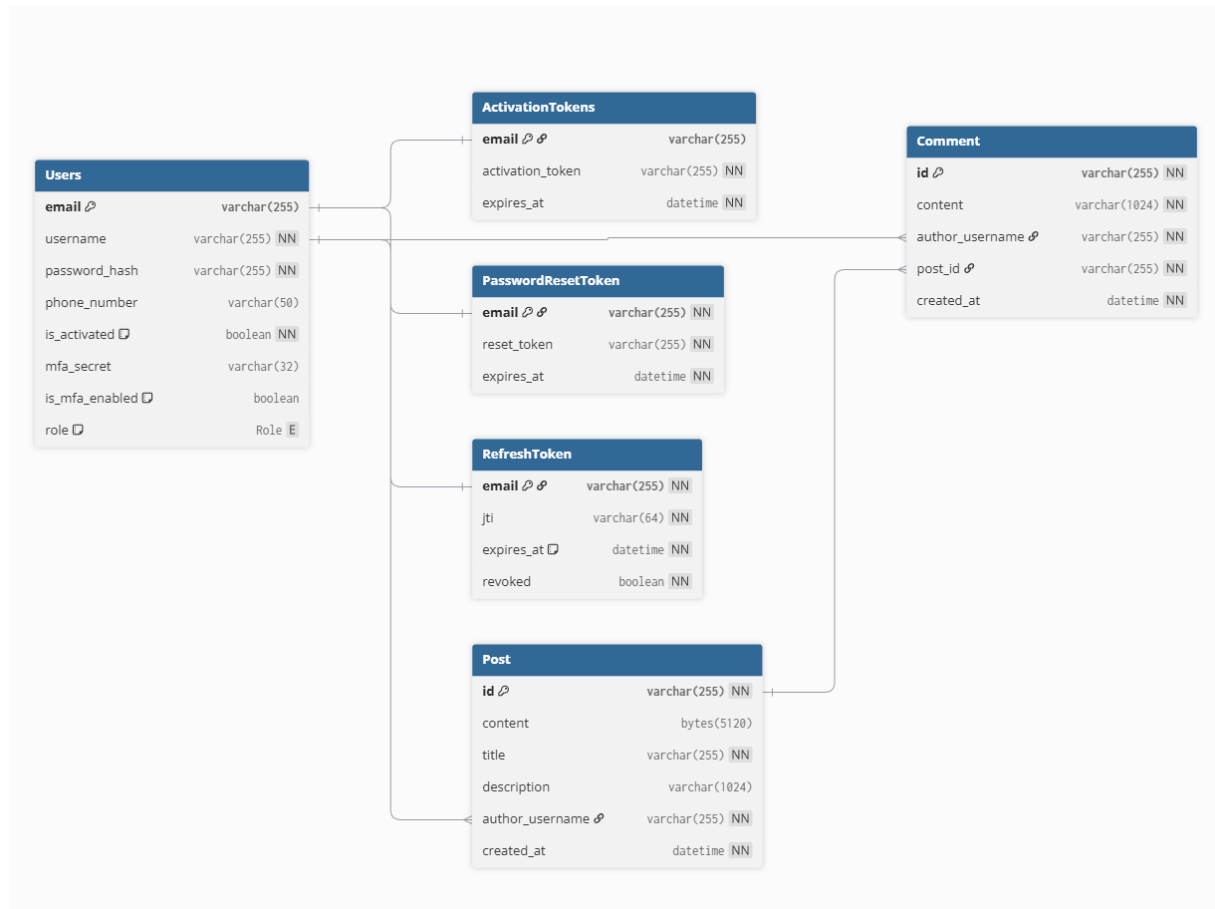


Figure 2: ER diagram

5.0 UML Sequence Diagrams

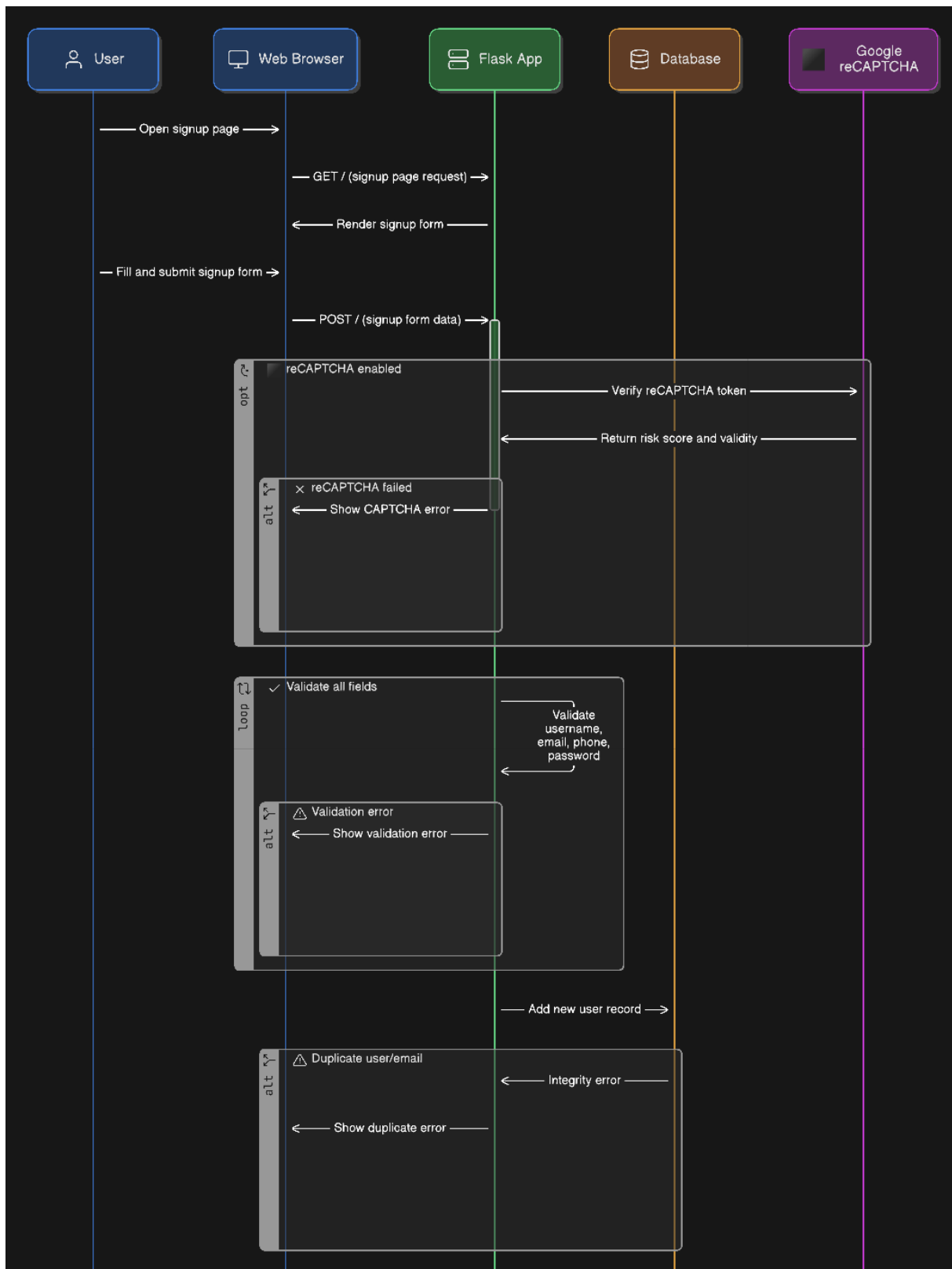


Figure 3: User Registration and Account Activation Process (part1)

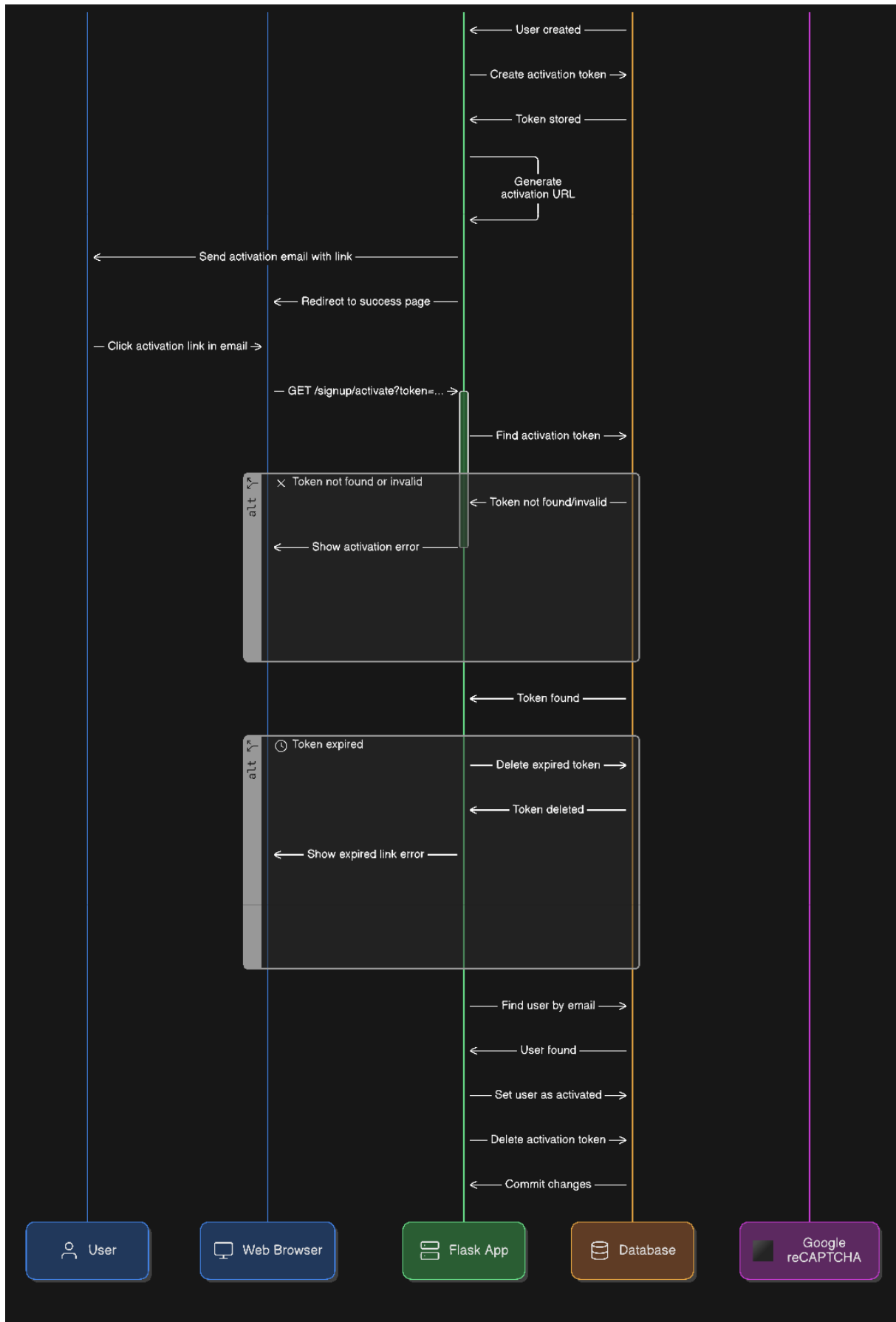


Figure 4: User Registration and Account Activation Process (part2)

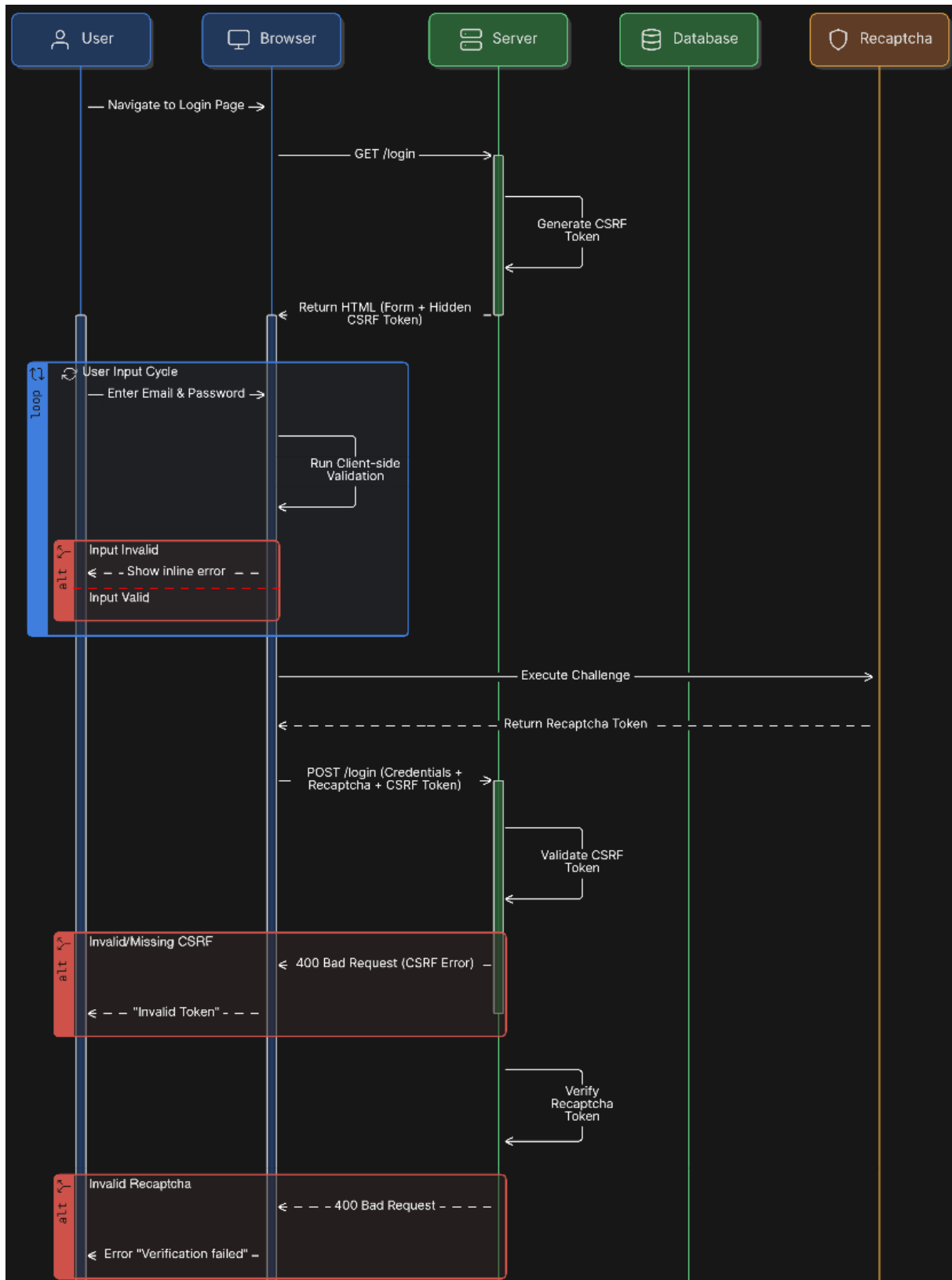


Figure 5: User login process (Part 1)

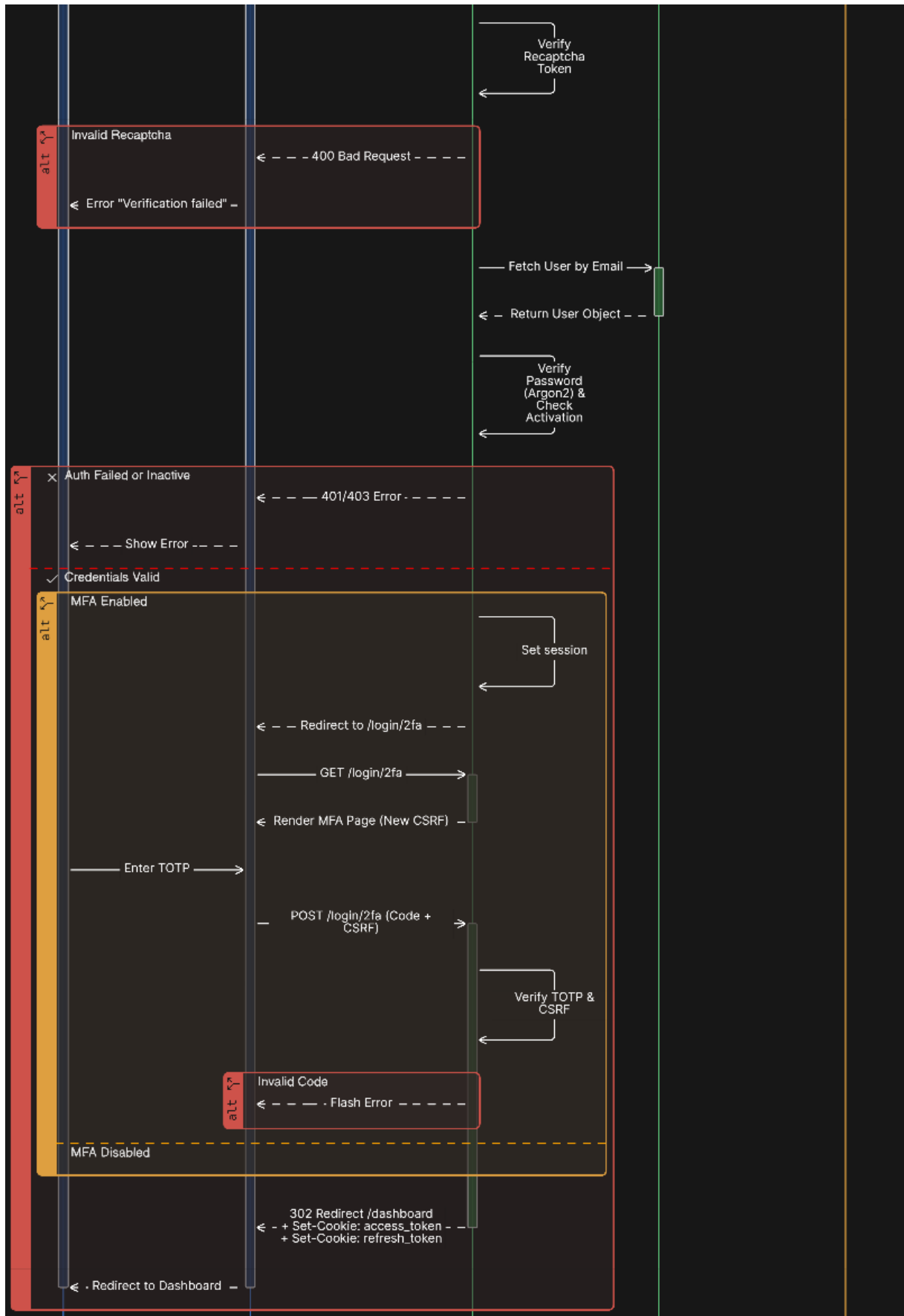


Figure 6: User login process (Part 2)

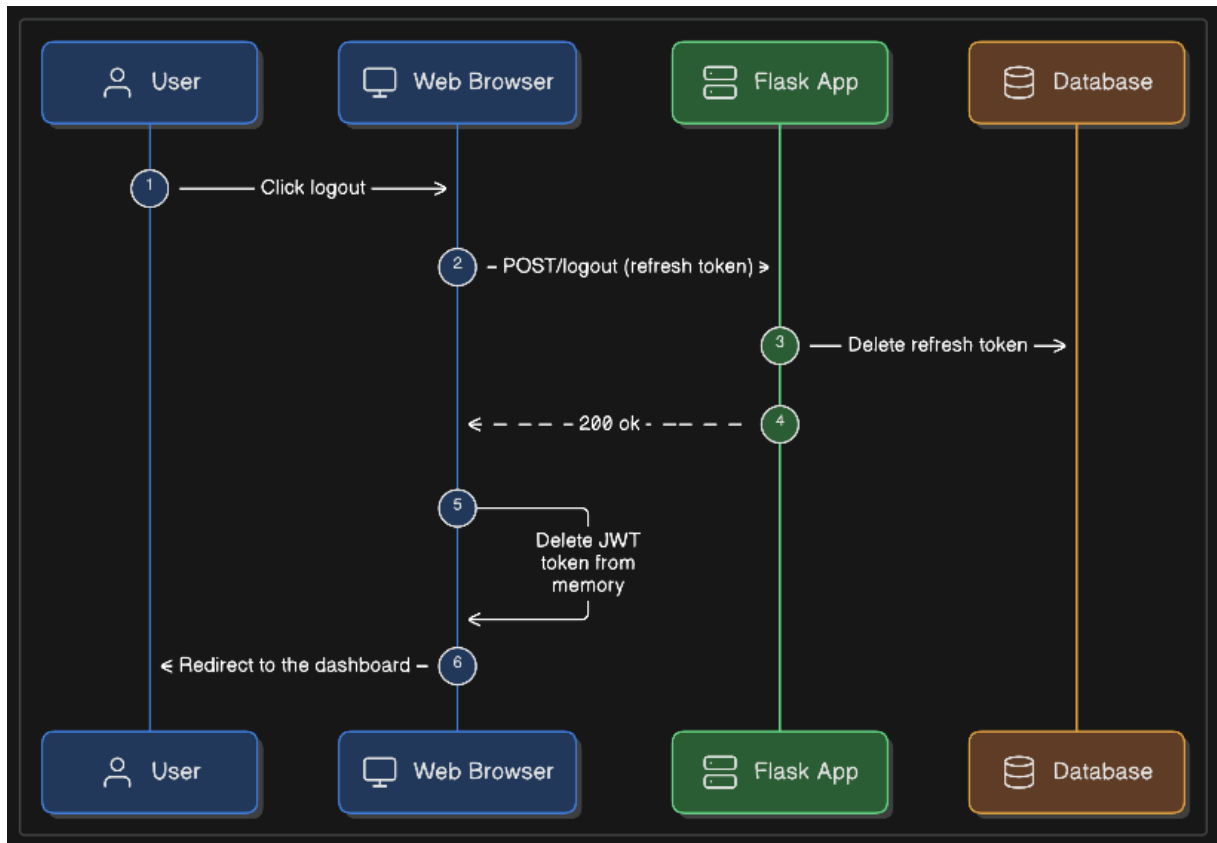


Figure 7: User logout process

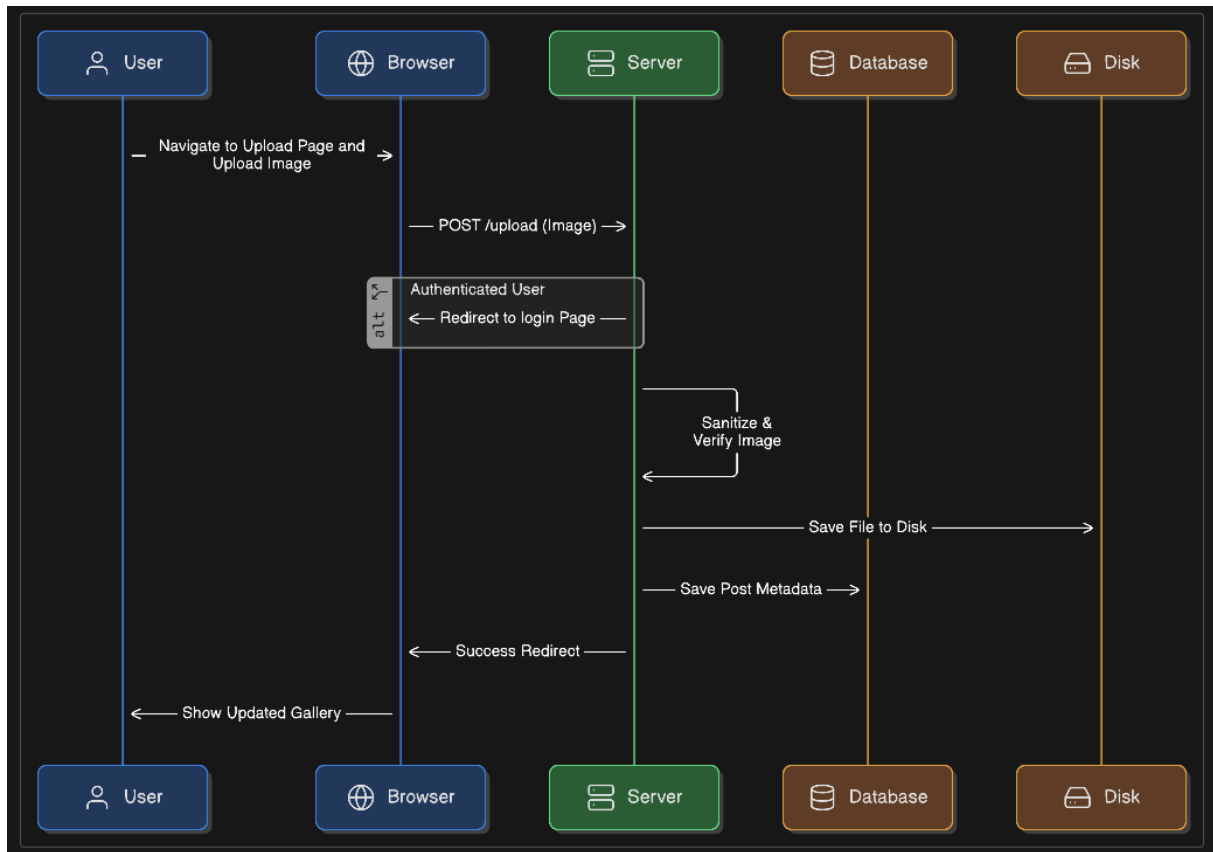


Figure 8: Content upload process

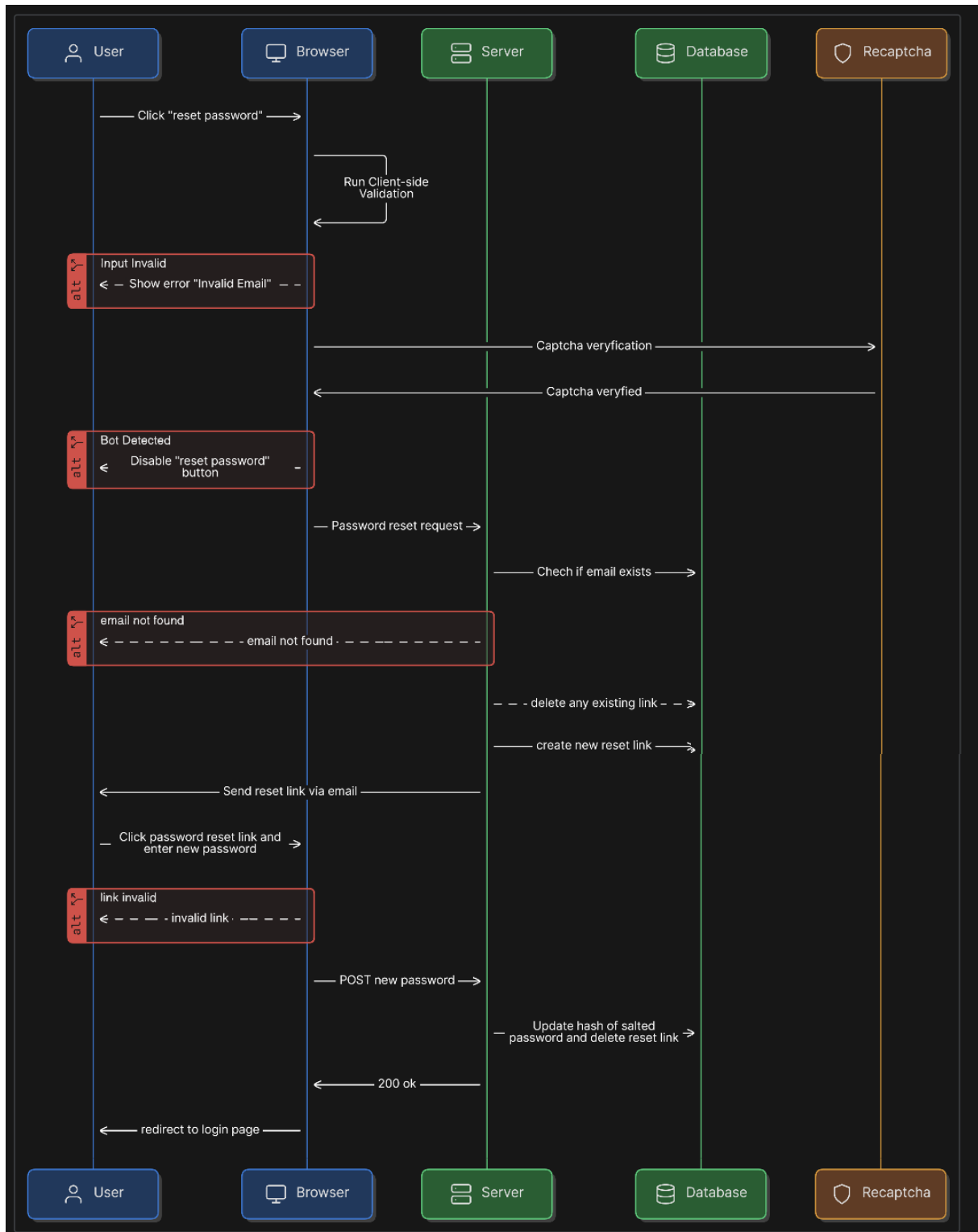


Figure 9: Reset password process

6.0 Explanations of key implementation choices

- **Environment-Based Configuration:** All critical application settings are loaded from environment variables via dotenv
- **Hashing Password with Pepper:** Adds an application-wide secret to passwords before hashing

-
- **reCaptcha:** Mitigates automated signup attacks and bot activity
 - **Domain Restriction Controls:** Whitelist and blacklist support for email domains
 - **Invalid Token Handling:** Handles invalid or expired tokens
 - **Activation Token Expiry and Token Hashing:** Mitigates risk of stale activation links being reused or intercepted
 - **Validation of Username, Password, Email Address and Phone Number:** Validates all of the input values are correct, don't contain bad characters and follow the security rules
 - **Optional Two-Factor Authentication (2FA):** Implements Time-based One-Time Passwords (TOTP) and QR codes, requiring users to provide a second form of verification during login
 - **CSRF Protection:** Enforces Cross-Site Request Forgery tokens on all state-changing requests (POST), preventing malicious sites from acting on behalf of logged-in users

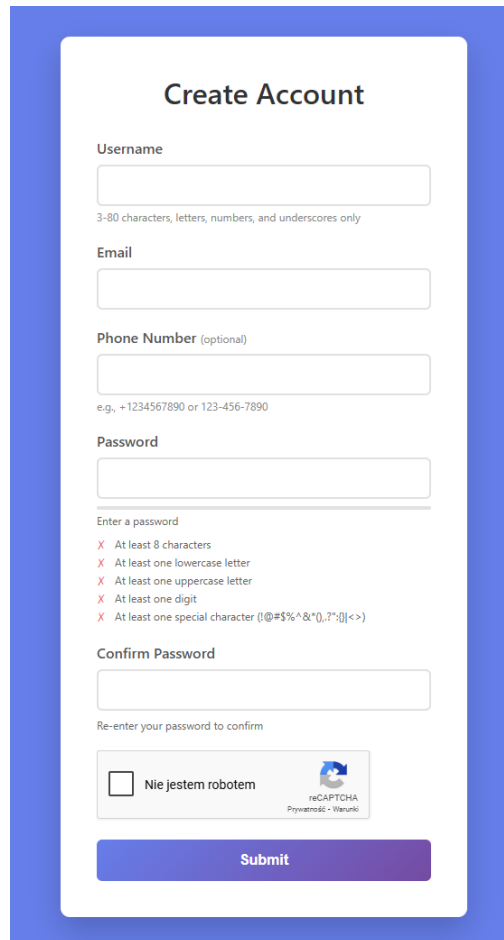
7.0 Description of security mechanisms

- **Argon2id Password Hashing:** Modern, memory-hard password hashing algorithm with salting
- **Password Pepper:** Optional server-side secret for additional security
- **Password Policy:** Enforces strong passwords (8+ chars, mixed case, digit, special char)
- **reCAPTCHA:** Google reCAPTCHA v2 to prevent automated registration abuse
- **Email Domain Restrictions:** Whitelist/blacklist email domains Account Activation: Time-limited activation tokens prevent unverified accounts
- **Token Security:** Activation tokens are hashed (SHA256) in database, expire after 24 hours
- **HTTPS Enforcement:** Flask-Talisman with HSTS headers (optional)
- **Input Validation:** Server-side validation for all inputs
- **SQL Injection Prevention:** SQLAlchemy ORM
- **XSS Protection:** Jinja2 auto-escaping
- **CSRF Protection:** Flask-WTF enforces cryptographic tokens on all state-changing requests
- **HTTPS Enforcement:** Redirects HTTP traffic to HTTPS
- **Multi-Factor Authentication:** Implements Time-based One-Time Passwords, requiring a second verification step using Google Authenticator
- **Role Base Access Control:**
 - **Guest users (unauthenticated):** can browse public content and use keyword-based search
 - **Registered users (authenticated):** can add new content (including file uploads), comment / rate posts, and delete only their own content

-
- **Administrators (authenticated)**: can moderate the platform by deleting or managing any content item or comment
 - **Owners (authenticated)**: have all the administrators' permissions and can manage users' roles

Role is verified during request processing. To ensure that role demotion/promotion is taken into account in the real time, the role is verified by fetching user data from the database.

8.0 Screenshots



The screenshot shows a 'Create Account' form with the following fields and validation rules:

- Username**: A text input field with a validation message: '3-80 characters, letters, numbers, and underscores only'.
- Email**: A text input field.
- Phone Number (optional)**: A text input field with a validation message: 'e.g., +1234567890 or 123-456-7890'.
- Password**: A text input field with a validation message: 'Enter a password'.
- Confirm Password**: A text input field with a validation message: 'Re-enter your password to confirm'.

Below the password fields, there is a list of validation rules for the password:

- X At least 8 characters
- X At least one lowercase letter
- X At least one uppercase letter
- X At least one digit
- X At least one special character (!@#\$%^&*()_?~|}|<>)

At the bottom of the form, there is a checkbox labeled 'Nie jestem robotem' (I am not a robot) and a reCAPTCHA widget. Below these is a purple 'Submit' button.

Figure 10: Registration form

Create Account

Username

username1

3-80 characters, letters, numbers, and underscores only

Email

username1@example.com

Phone Number (optional)

+48123123125

e.g., +1234567890 or 123-456-7890

Password

Password123!

Strong password

✓ At least 8 characters

✓ At least one lowercase letter

✓ At least one uppercase letter

✓ At least one digit

✓ At least one special character (!@#\$%^&*(),.?~:|[]<>)

Confirm Password

Re-enter your password to confirm

☐ Nie jestem robotem

reCAPTCHA

Przywróć • Wrażliwość

Submit

Figure 11: Valid registration data

✓

Registration Successful!

Your account has been created successfully.

Check your email to activate your account.

Didn't receive the email? Check your spam folder or contact support.

Back to Signup

Figure 12: Successful registration

15

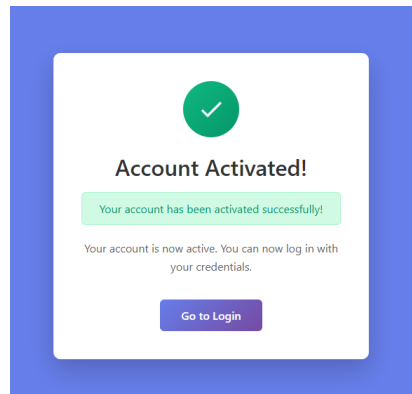


Figure 13: Successful activation

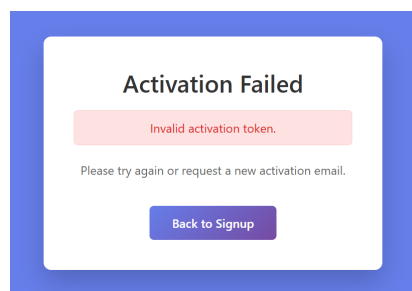


Figure 14: Invalid activation

A screenshot of a login page. The title "Login" is centered at the top. Below it, there are two input fields: "Email" and "Password". Below the password field, there is a reCAPTCHA widget with the text "Nie jestem robotem" and a checkbox. To the right of the checkbox is the reCAPTCHA logo and the text "reCAPTCHA Prywatność • Warunki". Below the reCAPTCHA widget, there is a purple button labeled "Login". At the bottom, there are two links: "Don't have an account? Sign Up" and "Forgot your password? Reset Password". The entire form is centered within a white box with rounded corners, set against a solid blue background.

Figure 15: Login page

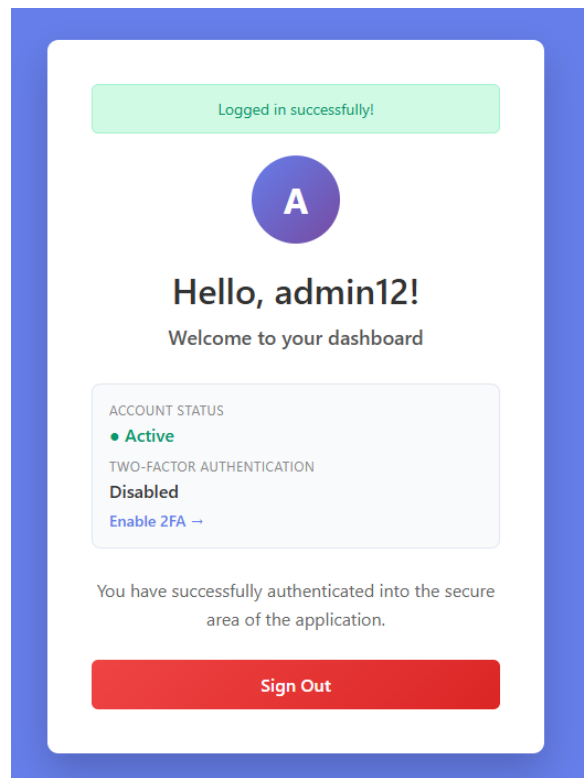


Figure 16: Dashboard



Figure 17: MFA setup

Authentication

Please enter the code from your authenticator app.

0 0 0 0 0 0

Verify

[Back to Login](#)

Figure 18: MFA

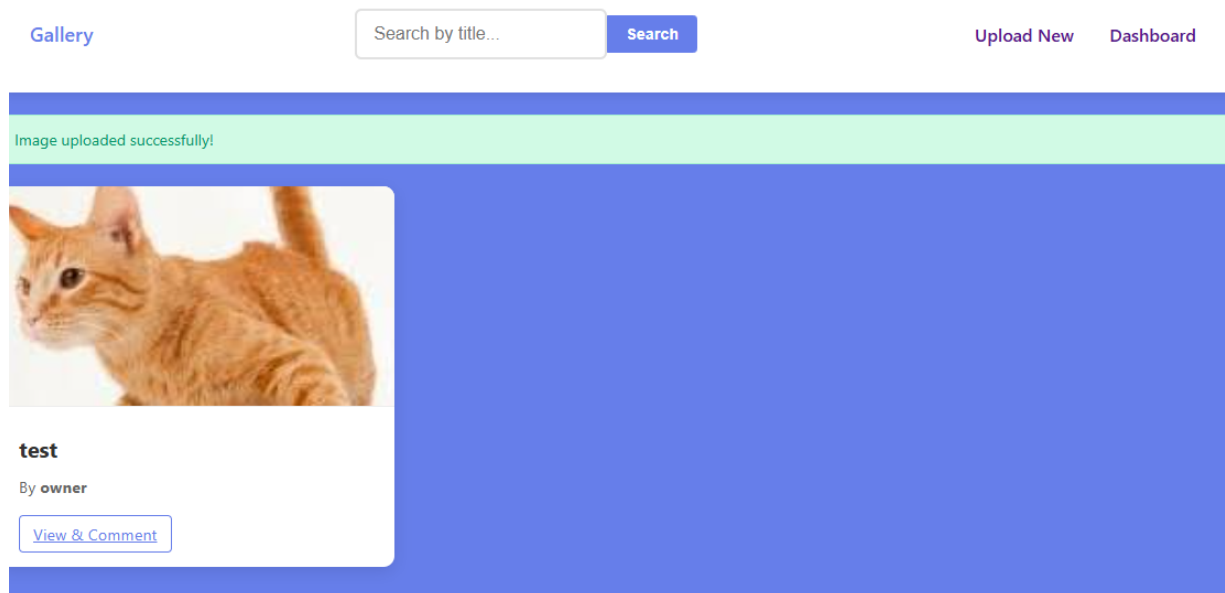
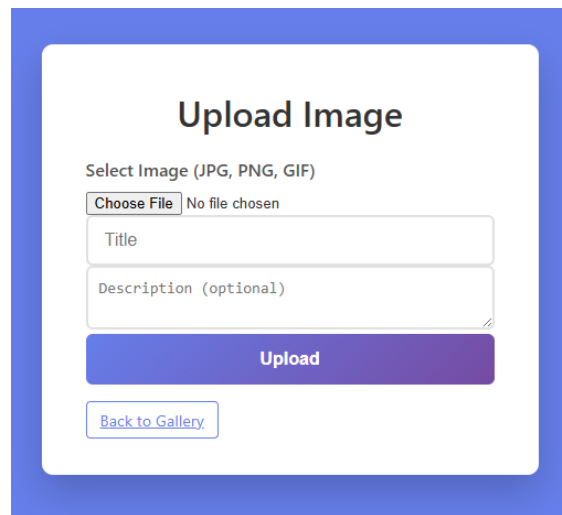
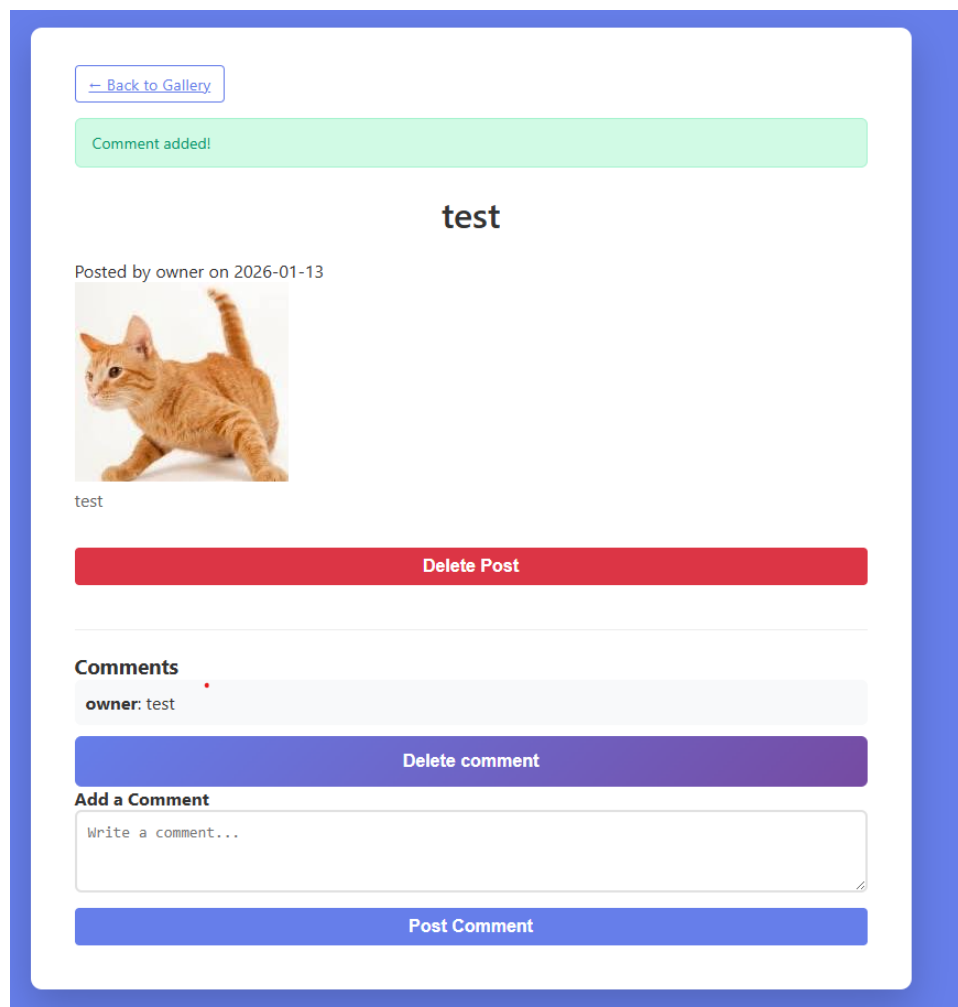


Figure 19: Gallery



The screenshot shows a form titled "Upload Image" within a blue border. The form is white and contains the following elements: a heading "Upload Image", a label "Select Image (JPG, PNG, GIF)", a "Choose File" button next to the text "No file chosen", a "Title" text input field, a "Description (optional)" text input field, a large blue "Upload" button, and a "Back to Gallery" button at the bottom.

Figure 20: Post creation with image upload



The screenshot shows a post details page within a blue border. At the top left is a "Back to Gallery" button. Below it is a green notification bar that says "Comment added!". The post title is "test". Below the title, it says "Posted by owner on 2026-01-13". There is an image of an orange cat. Below the image is the text "test". A red "Delete Post" button is located below the post content. A "Comments" section follows, showing a comment by "owner: test" with a "Delete comment" button. Below the comments is an "Add a Comment" section with a text input field containing "Write a comment..." and a "Post Comment" button.

Figure 21: Post details

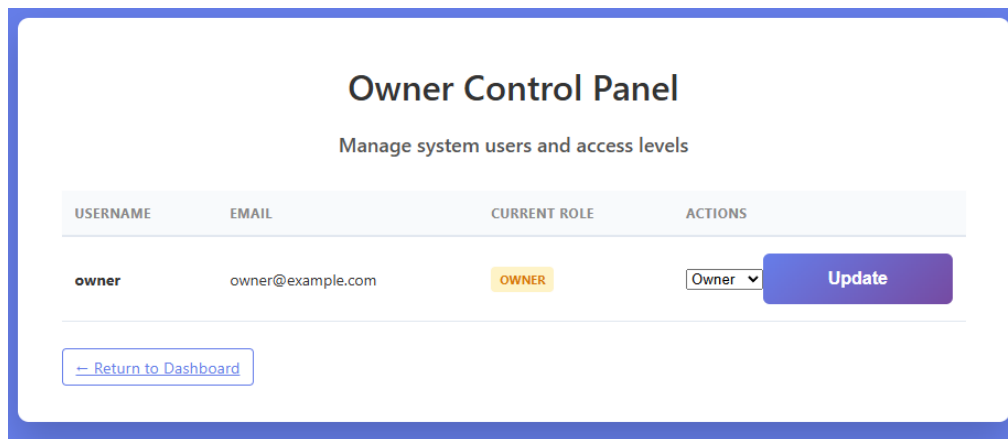


Figure 22: Owner panel

9.0 Conclusion

This project implements a secure and maintainable user signup system using Flask and SQLAlchemy, with strong emphasis on security and validation. It features robust input validation, Argon2id-hashed passwords with a pepper, email activation using hashed tokens with expiration time, and bot protection via Google reCAPTCHA Enterprise. Configuration is environment-based for flexibility across development and production, and HTTPS enforcement can be enabled for secure communication. Overall, it provides a production-ready foundation for safe user registration with a modular architecture that is easy to extend.